

EQ2330 Image and Video Processing

Project 2

Chong, Ngai Chun Adrian
ncachong@kth.se

Oğuzhan Oğuz
oguzhano@kth.se

December 7, 2025

Summary

This project explores the distortion-rate performance of image compression algorithms using Discrete Cosine Transform (DCT) and Fast Wavelet Transform (FWT) with 5/3 biorthogonal filter banks. Implemented on test images (boats, harbour, peppers), the systems apply blockwise 8x8 DCT or multi-scale FWT, followed by uniform mid-tread quantization (step sizes 2^0 to 2^9) and entropy-based bit-rate estimation via ideal variable-length coding.

Results show similar performance between DCT and FWT, with PSNR improving linearly at higher bit-rates, approaching the Gaussian distortion-rate bound. Harbour image exhibits the lowest PSNR due to high-frequency details, while boats and peppers perform better. Distortion (MSE) equals in image and transform domains for orthonormal DCT but differs for biorthogonal 5/3 filters, verifying Parseval's theorem.

1 Introduction

This project investigates the achievable distortion-rate performance of discrete cosine transform (DCT) and fast wavelet transform (FWT) based image compression algorithms respectively.

In this report, section 2 describes the implementation of DCT, FWT, quantization, and bit-rate estimation steps; and section 3 compares the performance and resultant distortion-rate curves of each algorithm.

2 System Description

Both DCT and FWT based image compression algorithms follow the same structure:

1. Apply transformation
2. Quantization
3. Variable-length coding and decoding
4. Dequantization
5. Apply inverse transformation

In this project, the actual coding and decoding in step 3 is omitted and an ideal average codeword length is derived directly as described in section 2.4.

2.1 DCT-based Image Compression

In this project, we use a blockwise 8×8 DCT-II transform, which is orthonormal and separable. In particular, the one-dimensional transformation matrix A is given by

$$a_{ik} = \alpha_i \cos\left(\frac{(2k+1)i\pi}{16}\right) \quad \text{for } i, k = 0, \dots, 7 \quad (1)$$

where

$$\alpha_0 = \sqrt{\frac{1}{8}}, \quad \alpha_i = \sqrt{\frac{1}{4}} \quad \text{for } i > 0$$

The complete 2-dimensional transform for each 8×8 block $\tilde{x}$ in the image x is then given by

$$\tilde{y} = A\tilde{x}A^T \quad (2)$$

and the inverse transform is

$$\tilde{x} = A^T\tilde{y}A \quad (3)$$

The computed matrix A is

$$A = \begin{bmatrix} 0.3536 & 0.3536 & 0.3536 & 0.3536 & 0.3536 & 0.3536 & 0.3536 & 0.3536 \\ 0.4904 & 0.4157 & 0.2778 & 0.0975 & -0.0975 & -0.2778 & -0.4157 & -0.4904 \\ 0.4619 & 0.1913 & -0.1913 & -0.4619 & -0.4619 & -0.1913 & 0.1913 & 0.4619 \\ 0.4157 & -0.0975 & -0.4904 & -0.2778 & 0.2778 & 0.4904 & 0.0975 & -0.4157 \\ 0.3536 & -0.3536 & -0.3536 & 0.3536 & 0.3536 & -0.3536 & -0.3536 & 0.3536 \\ 0.2778 & -0.4904 & 0.0975 & 0.4157 & -0.4157 & -0.0975 & 0.4904 & -0.2778 \\ 0.1913 & -0.4619 & 0.4619 & -0.1913 & -0.1913 & 0.4619 & -0.4619 & 0.1913 \\ 0.0975 & -0.2778 & 0.4157 & -0.4904 & 0.4904 & -0.4157 & 0.2778 & -0.0975 \end{bmatrix}$$

Figure 1 shows an example where a portion of the *boats* image and the absolute DCT coefficients in log-scale.

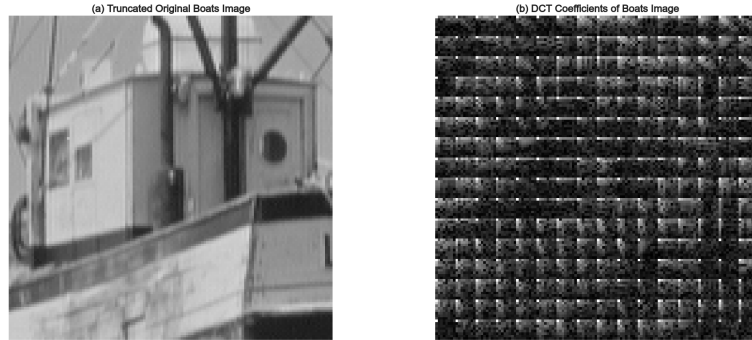


Figure 1: Original *boats* image and absolute DCT coefficients in log-scale

2.2 FWT-based Image Compression

2.2.1 Two-Band Filter Bank and Lifting Implementation

The core of the FWT coder is the one-dimensional two-band analysis and synthesis filter banks. In this project we employ the *lifting implementation* of the 5/3 biorthogonal wavelet filter bank, as illustrated in Fig. 1 of the project description.

Given a discrete-time signal $x[n]$ of even length, we first split it into even and odd samples

$$e_k = x[2k], \quad o_k = x[2k+1], \quad k = 0, \dots, \frac{N}{2} - 1. \quad (4)$$

The analysis lifting steps consist of a *predict* and an *update* stage. Using periodic extension at the boundaries, the high-pass (detail) coefficients d_k are obtained as

$$d_k = o_k - \frac{1}{2}(e_k + e_{k+1}), \quad (5)$$

where indices are taken modulo $N/2$. The low-pass (approximation) coefficients s_k are then computed by

$$s_k = e_k + \frac{1}{4}(d_{k-1} + d_k). \quad (6)$$

Finally, a simple normalization step is applied to balance the gains of the subbands. In our implementation we use

$$c_k^{(L)} = \sqrt{2} s_k, \quad c_k^{(H)} = \frac{1}{\sqrt{2}} d_k, \quad (7)$$

where $c_k^{(L)}$ and $c_k^{(H)}$ denote the low- and high-band coefficients, respectively. The output vector of the analysis filter bank is obtained by concatenating the two subbands, so that the total number of coefficients equals the input signal length.

The synthesis filter bank inverts these steps in reverse order. After undoing the normalization,

$$s_k = \frac{1}{\sqrt{2}} c_k^{(L)}, \quad d_k = \sqrt{2} c_k^{(H)}, \quad (8)$$

the inverse update and predict steps reconstruct the even and odd samples:

$$e_k = s_k - \frac{1}{4}(d_{k-1} + d_k), \quad (9)$$

$$o_k = d_k + \frac{1}{2}(e_k + e_{k+1}). \quad (10)$$

Interleaving e_k and o_k yields the reconstructed signal $\hat{x}[n]$. Due to the perfect reconstruction property of the 5/3 biorthogonal filter bank, the overall analysis-synthesis cascade satisfies

$$\hat{x}[n] \approx x[n] \quad \text{for all } n, \quad (11)$$

which we verified numerically by measuring the maximum absolute reconstruction error.

To handle border effects that occur during filtering, we employ periodic extension of the signal such that indices are taken modulo the signal length. This avoids artificial edge artifacts and guarantees that the number of output coefficients equals the number of input samples.

2.2.2 Two-Dimensional FWT

The two-dimensional FWT is implemented by applying the one-dimensional filter banks separably to the rows and columns of the image. Let $f(i, j)$ denote the input image of size $M \times N$, where M and N are powers of two. At each decomposition level:

1. The 1D analysis filter bank is applied to every row of the current approximation subband, producing low- and high-frequency components along the horizontal direction.
2. The same 1D analysis filter bank is then applied to every column of the intermediate result, yielding four subbands: LL, LH, HL and HH.
3. For the next scale, only the LL subband is further decomposed.

We perform this procedure recursively for a total of four scales. The resulting wavelet coefficients are stored in-place in a single matrix whose size equals that of the original image.

Figure 2 shows the logarithmic magnitude of the wavelet coefficients for the *Harbour* image at scale 4. The multi-resolution structure is clearly visible. The largest block in the top-left corner corresponds to the LL subband, which contains most of the image energy. The remaining blocks represent horizontal, vertical and diagonal detail information at different scales.

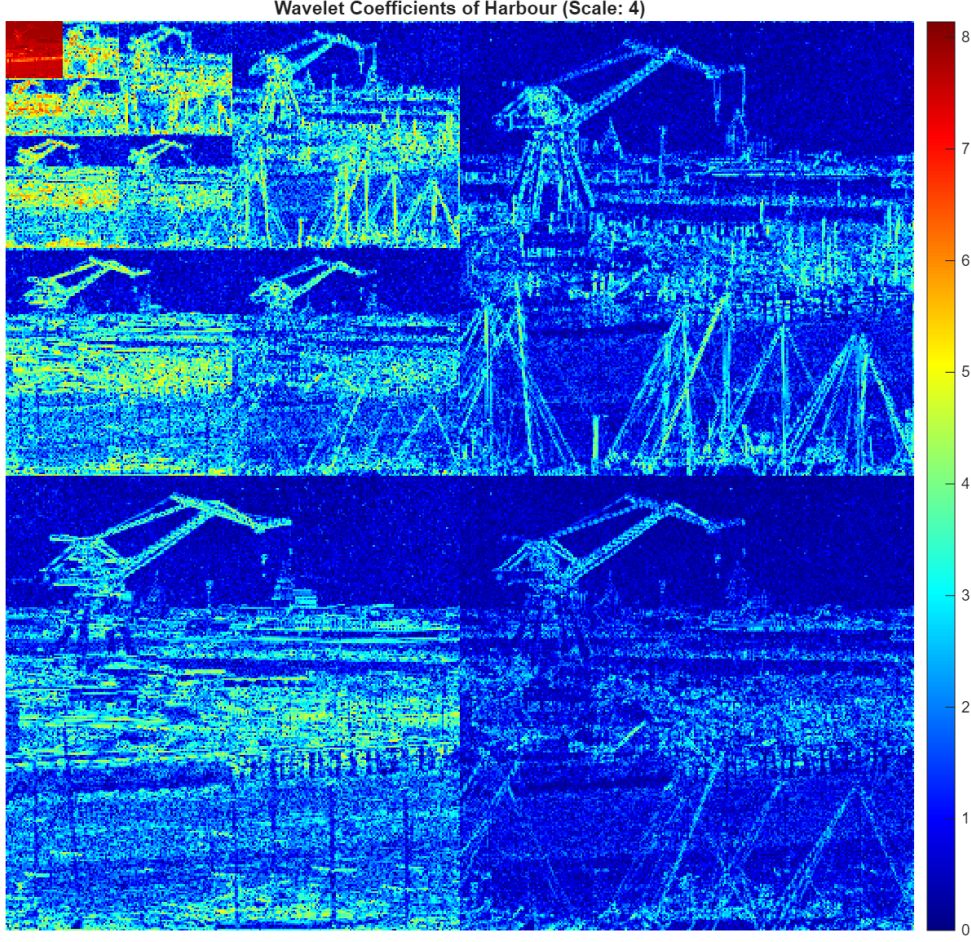


Figure 2: Wavelet Coefficients of Harbour (Scale: 4).

2.3 Uniform Quantization

After apply DCT or FWT, all coefficients are quantized by a uniform mid-tread quantizer without a dead-zone. For a coefficient x and step size Δ , the quantization index q and the reconstructed value $\hat{x}$ are given by

$$q = \text{round}\left(\frac{x}{\Delta}\right), \quad \hat{x} = \Delta q. \quad (12)$$

The same step size is used for all coefficients, and we vary Δ over the range

$$\Delta \in \{2^0, 2^1, \dots, 2^9\}. \quad (13)$$

Figure 3 shows a plot of the quantization function q against x/Δ in equation (12).

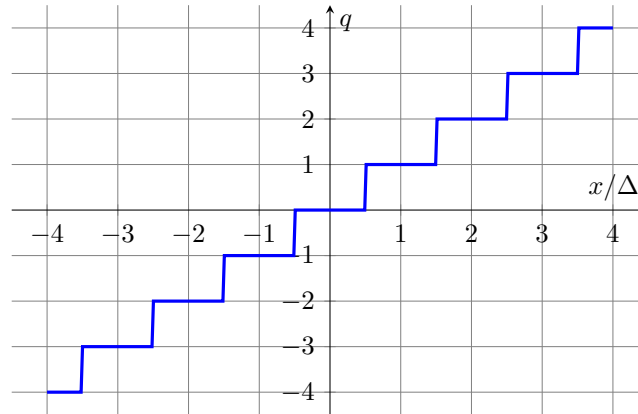


Figure 3: Quantization function in equation (12)

2.4 Bit-Rate Estimation

The estimated bit-rate needed to encode quantized DCT coefficients are given by an ideal variable-length code (VLC) where each of the 64 coefficients in a block are encoded separately. For each coefficient index $i \in \{1, \dots, 64\}$, we compute the empirical probability mass function $p_i(m)$ across all blocks and the entropy

$$H_i = - \sum_m p_i(m) \log_2 p_i(m) \quad [\text{bits/coeff}]. \quad (14)$$

The total bit-rate in bits per pixel is then given by

$$R = \frac{1}{64} \sum_{i=1}^6 4H_i \quad (15)$$

Similarly, to estimate the bit-rate required to encode the quantized subband coefficients, we assume an ideal VLC that encodes each subband independently. For a given subband s with N_s coefficients, we compute the empirical probability mass function $p_s(m)$ of the quantized levels and evaluate the entropy

$$H_s = - \sum_m p_s(m) \log_2 p_s(m) \quad [\text{bits/coeff}]. \quad (16)$$

The total bit-rate in bits per pixel is then approximated as

$$R = \frac{1}{MN} \sum_s N_s H_s, \quad (17)$$

where the sum runs over all subbands.

In accordance with the project instructions, the same set of VLCs is used across the three test images *boats*, *harbour* and *peppers*. For each step size, an estimated bit-rate is produced by considering the combined statistics of all three images.

3 Results

3.1 Distortion Measures

The quality of the reconstructed images is quantified using the Peak Signal-to-Noise Ratio (PSNR). For 8-bit images, the PSNR is defined as

$$\text{PSNR} = 10 \log_{10} \left(\frac{255^2}{d} \right) \quad [\text{dB}], \quad (18)$$

where d is the distortion measured with Mean Squared Error (MSE) between the original image $f(i, j)$ and the reconstructed image $\hat{f}(i, j)$:

$$d = \frac{1}{MN} \sum_{i=0}^{M-1} \sum_{j=0}^{N-1} (f(i, j) - \hat{f}(i, j))^2. \quad (19)$$

In addition, we compute the MSE in the DCT and wavelet domain,

$$d_y = \frac{1}{MN} \sum_k (y_k - \tilde{y}_k)^2, \quad d_w = \frac{1}{MN} \sum_k (w_k - \tilde{w}_k)^2, \quad (20)$$

where y_k and $\tilde{y}_k$ denote the original and quantized DCT coefficients, while w_k and $\tilde{w}_k$ denote the original and quantized wavelet coefficients, respectively.

Comparing the image-domain distortion d with the transform-domain distortions d_y (for the DCT) and d_w (for the 5/3 filter bank), we observe that $d = d_y$ for the DCT transform, while d and d_w are not equal (but strongly correlated) for the 5/3 filter bank. Table 1 provides exemplary distortion values for step size $\Delta = 2^2$.

Image	DCT		FWT (5/3 Filter Banks)	
	d	d_y	d	d_w
boats	1.2709	1.2709	1.5090	1.2629
harbour	1.1791	1.1791	1.3637	1.1588
peppers	1.3240	1.3240	1.6306	1.3321

Table 1: Distortion values d , d_y , d_w of each test image with quantization step size $\Delta = 2^2$

In general, Parseval’s theorem holds for orthonormal / unitary transforms, such as the normalized DCT-II transform and Daubechies-8. Therefore, energy preservation would imply $d = d_y$. While biorthogonal filters, such as 5/3 filter bank, does not guarantee energy preservation, so in general $d \neq d_w$.

3.2 Rate–Distortion Curves

For each step size Δ and for each of the three test images (*boats*, *harbour*, *peppers*), we compute the corresponding bit-rate R and PSNR. Figure 4 and Figure 5 illustrates the resulting rate–distortion curves using DCT-based compression and FWT-based compression at scale 4 respectively.

We observe that the *Harbour* image typically yields the lowest PSNR for a given bit-rate. This behavior is expected, since the image contains many high-frequency details and textures, which lead to larger quantization errors in the high-frequency subbands. In contrast, the *Boats* and *Peppers* images have larger smooth regions and fewer fine details, which results in a higher PSNR at the same bit-rate.

Comparing the distortion-rate curves of our implementations of DCT-based and FWT-based image compression, we observe that both methods results in similar performances, especially at higher bit-rates. As we increase the bit-rate, the PSNR becomes linear to bit-rate and the curves for each image converges to the combined mean curve, approaching the theoretical Shannon lower bound of the Gaussian $D(R)$ curve.

An overall rate–PSNR curve for the complete data set can be obtained by averaging the rates and PSNR values over the three images for each step size. This averaged curve summarizes the expected performance of the DCT-based and FWT-based coders on this class of images.

4 Conclusions

In our limited testing, both image compression algorithms based on DCT and FWT using 5/3 biorthogonal filter banks show similar rate-distortion performances. Therefore, thorough testing across more images would be needed to differentiate the two. However, both algorithms exhibit the same behavior when increasing bit-rate — the distortion-rate curve approaches the theoretical Gaussian $D(R)$ curve bound. Across the three test images, both algorithms show similar efficiency variations due to the characteristics for each image’s content.

Furthermore, by comparing the distortion measured by mean square error in the image and transform domains respectively, we have verified that both values agree for orthonormal / unitary transforms due to their energy preservation property and Parseval’s theorem.

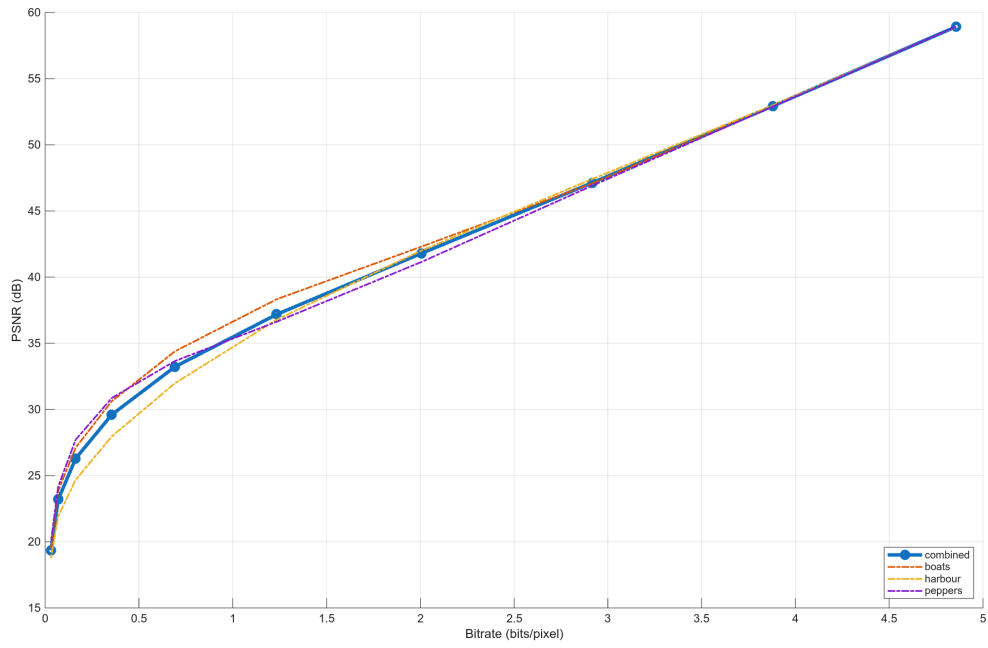


Figure 4: DCT Rate-Distortion for the three test images.

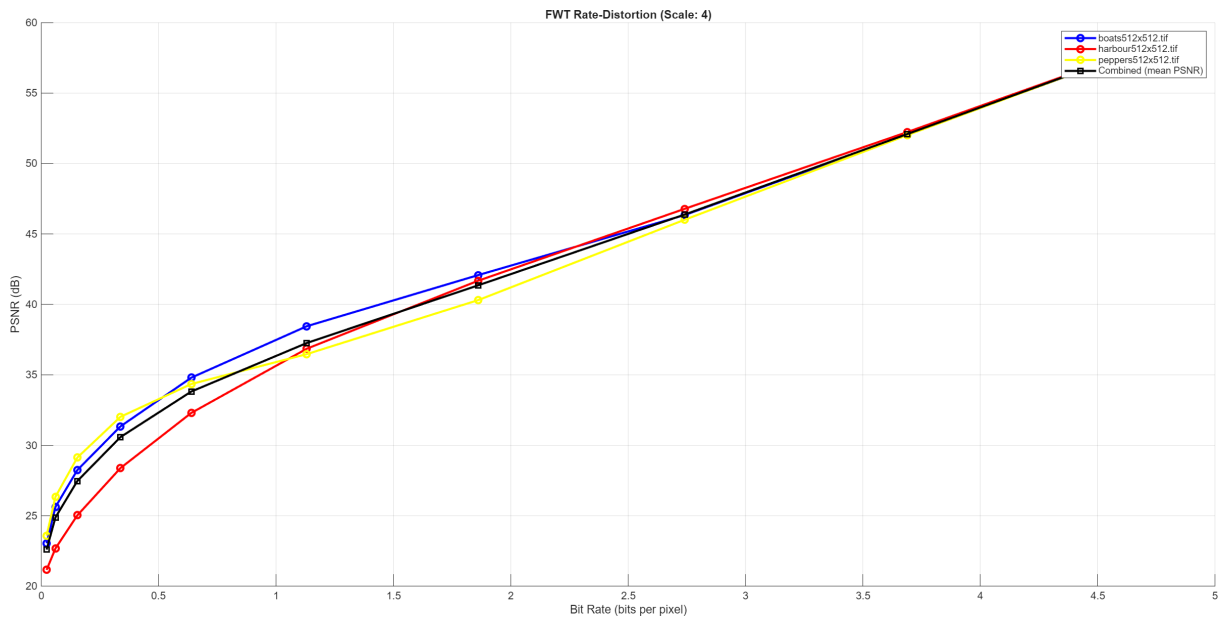


Figure 5: FWT Rate-Distortion for the three test images.

Appendix

Who Did What

The project is divided into two parts: Chong was responsible for the implementation of the DCT-based compression algorithm, while Oguz was responsible for the FWT-based compression algorithm. After completion, the results were synthesized and feedback was provided after reviewing the entire report.

MatLab code

```
%%% DCT %%%

function A = dct_A(M)
% compute matrix A for MxM DCT
A = zeros(M);
for ii = 1:M
    for kk = 1:M
        if ii == 1
            A(ii,kk) = sqrt(1/M) * cos((2*kk-1) * (ii-1) * pi / 2 / M);
        else
            A(ii,kk) = sqrt(2/M) * cos((2*kk-1) * (ii-1) * pi / 2 / M);
        end
    end
end
end

function f_dct = dct(f, M)
% perform blockwise MxM DCT on f
A = dct_A(M);
[w,h] = size(f);
f_dct = zeros(size(f));
for x = 1:M:w
    for y = 1:M:h
        f_dct(x:x+M-1,y:y+M-1) = A * f(x:x+M-1,y:y+M-1) * A';
    end
end
end

function f_recon = inv_dct(f, M)
% perform blockwise MxM inverse DCT on f
A = dct_A(M);
[w,h] = size(f);
f_recon = zeros(size(f));
for x = 1:M:w
    for y = 1:M:h
        f_recon(x:x+M-1,y:y+M-1) = A' * f(x:x+M-1,y:y+M-1) * A;
    end
end
end

function [mean_cw_len, cw_len] = ideal_cw_len(f,M)
% estimate bit-rate by using ideal average code word lengths
cw_len = zeros(M);
for x = 1:M
    for y = 1:M
        coeff = f(x:M:end,y:M:end);
        cw_len(x,y) = shannon_entropy(reshape(coeff,[],1));
    end
end
end
```



```

mean_cw_len = mean2(cw_len);
end

%%% FWT %%%

function [y] = fwt_analysis_1d(x)
% 1D forward 5/3 lifting-based wavelet transform (analysis)
    x = x(:)';
    N = length(x);

    even = x(1:2:end);
    odd = x(2:2:end);

    even_next = circshift(even, [0, -1]);

    high = odd - 0.5 * (even + even_next);

    high_prev = circshift(high, [0, 1]);

    low = even + 0.25 * (high + high_prev);

    low = low * sqrt(2);
    high = high / sqrt(2);

    y = [low, high];
end

function [x_recon] = fwt_synthesis_1d(y)
% 1D inverse 5/3 lifting-based wavelet transform (synthesis)
    y = y(:)';
    N = length(y);
    half = N / 2;

    low = y(1:half);
    high = y(half+1:end);

    low = low / sqrt(2);
    high = high * sqrt(2);

    high_prev = circshift(high, [0, 1]);
    even = low - 0.25 * (high + high_prev);

    even_next = circshift(even, [0, -1]);
    odd = high + 0.5 * (even + even_next);

    x_recon = zeros(1, N);
    x_recon(1:2:end) = even;
    x_recon(2:2:end) = odd;
end

function [coeffs] = fwt_2d_analysis(img, scale)
% 2D separable forward FWT (row/column passes, given number of scales)
    [R, C] = size(img);
    coeffs = double(img);

    current_rows = R;

```

```

current_cols = C;

for s = 1:scale
    for r = 1:current_rows
        row = coeffs(r, 1:current_cols);
        coeffs(r, 1:current_cols) = fwt_analysis_1d(row);
    end

    for c = 1:current_cols
        col = coeffs(1:current_rows, c)';
        coeffs(1:current_rows, c) = fwt_analysis_1d(col)';
    end

    current_rows = current_rows / 2;
    current_cols = current_cols / 2;
end
end

function [img_recon] = fwt_2d_synthesis(coeffs, scale)
% 2D separable inverse FWT (row/column passes, given number of scales)
[R, C] = size(coeffs);
img_recon = double(coeffs);

for s = scale:-1:1
    current_rows = R / (2^(s-1));
    current_cols = C / (2^(s-1));

    for c = 1:current_cols
        col_coeffs = img_recon(1:current_rows, c)';
        img_recon(1:current_rows, c) = fwt_synthesis_1d(col_coeffs)';
    end

    for r = 1:current_rows
        row_coeffs = img_recon(r, 1:current_cols);
        img_recon(r, 1:current_cols) = fwt_synthesis_1d(row_coeffs);
    end
end
end

function [q_coeffs] = uniform_quantizer(coeffs, step_size)
% Mid-rise uniform scalar quantizer with step size 'step_size'
q_indices = round(coeffs / step_size);

q_coeffs = q_indices * step_size;
end

function H = calc_entropy(data)
% Empirical entropy (bits/sample) of input data
data = data(:);
if isempty(data)
    H = 0;
    return;
end

unique_vals = unique(data);

```

```

counts = histc(data, unique_vals);

probs = counts / sum(counts);

probs = probs(probs > 0);

H = -sum(probs .* log2(probs));
end

function total_bits = estimate_bitrate(coeffs, scale)
% Bit-rate estimate using per-subband entropies for a 'scale'-level FWT
[R, C] = size(coeffs);
total_bits = 0;

r_limit = R / (2^scale);
c_limit = C / (2^scale);

LL_band = coeffs(1:r_limit, 1:c_limit);
H_LL = calc_entropy(LL_band);
total_bits = total_bits + (numel(LL_band) * H_LL);

for s = scale:-1:1
    curr_r = R / (2^s);
    curr_c = C / (2^s);
    next_r = R / (2^(s-1));
    next_c = C / (2^(s-1));

    HL = coeffs(1:curr_r, curr_c+1:next_c);
    total_bits = total_bits + (numel(HL) * calc_entropy(HL));

    LH = coeffs(curr_r+1:next_r, 1:curr_c);
    total_bits = total_bits + (numel(LH) * calc_entropy(LH));

    HH = coeffs(curr_r+1:next_r, curr_c+1:next_c);
    total_bits = total_bits + (numel(HH) * calc_entropy(HH));
end
end

```

References

- [1] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, Prentice Hall, Upper Saddle River, New Jersey, second edition, 2002.
- [2] S. Mallat, *A Wavelet Tour of Signal Processing*, Academic Press, San Diego, California, second edition, 1999.